

Universidad Eafit - Ingeniería de sistemas

Organización de computadores 2026-1 - 25 de marzo de 2026

Proyecto 2 ALU y nuevos comandos a implementar en la ALU

Docente José Luis Montoya Pareja

Objetivo

Practicar los conceptos vistos en clase durante todo el semestre desde el diseño de una tabla de verdad, ALUs, construcción de la CPU y programación en assembler hasta su implementación en el HDL Nand2Tetris con circuitos adicionales a los que la plataforma Nand2Tetris trae.

Paso inicial

Cree en su repositorio de GIT la carpeta organizacionComputadores y dentro cree la carpeta proyecto2 y haga un commit inicial con la siguiente estructura:

```
[suUsuarioGit]/
├── organizacionComputadores/
│   ├── proyecto2/
│   ├── CONTRIBUTORS.md          # Colaboradores y roles
│   ├── CHANGELOG.md            # Historial de cambios
│   ├── LICENSE                  # Licencia del proyecto
│   └── README.md
```

En el archivo CONTRIBUTION.md se crea una tabla con la siguiente información:

Contribución de Miembros

Nombre	Rol	Tareas Asignadas	% Contrib
-----	-----	-----	-----
Ana Pérez	Líder	Coordinación, shell loop	20%
Luis Gómez	Dev	Comandos archivos, parser	25%
Carlos Ruiz	Dev	Comandos sistema, utils	25%
María López	Tester	Pruebas, depuración	20%
Pedro Díaz	Doc	Documentación, ejemplos	10%
			Total: 100%

En el archivo README.md incluya la siguiente información:

```
**Organización de computadores**
**Proyecto 2**
```

```

** 2026-1**
**Integrantes proyecto 2**
[Nombre alumno 1]
[Nombre alumno 2]

```

Punto # 1 - Circuito nuevo: Shifter (25%)

Herramientas a utilizar Aplicación web de Nand2Tetris

Descripción El circuito del Shifter tiene la siguiente descripción en formato HDL:

```

/*****
* Shifter.hdl - Desplazamiento de un bit a la izquierda o derecha
* Autor 1:
* Autor 2:
*
* Implementación del siguiente algoritmo:
* if (direction == 0){
*   out[0] = 0;
*   for(i=2; i < 16; i++)
*     out[i] = in[i-1];
* }
* else{
*   out[15] = 0;
*   for(i=1; i < 15; i++)
*     out[i] = in[i-1];
* }
*****/

```

```

CHIP Shifter {
    IN  in[16],          // Dato a correr
        direction;      // Dirección de corrimiento
                        // 0 = izquierda, 1 = derecha

    OUT out[16],         // Dato corrido en la dirección solicitada
        result;         // bit que sale por el extremo
                        // correspondiente

    PARTS:
    // Incluya su código aquí
}

```

Un cambio importante respecto circuito visto en clase es que esta especificación tiene un campo result, por el cual sale el bit que fue empujado en el extremo correspondiente. Para entender esto, mire la siguiente tabla:

Entrada direction	Entrada in[16] (ejemplo)	Salida out[16] (esperada)	Salida result
0	1000 0000 0000 0001	0000 0000 0000 0010	1
1	1000 0000 0000 0001	0100 0000 0000 0000	1

Tabla 1. Casos de prueba propuestos

En el caso donde direction es 0, el bit 15 (el más significativo) se lleva a result (por eso está en 1) y en el bit 0 aparece un 0. Lo inverso pasa cuando el bit direction es 1 (sale en result el bit 0 que estaba en 1 y en el bit 15 aparece un 0). El resto del número se corre un bit en la dirección indicada por direction.

Implementar en HDL el circuito que resuelve el shifter tanto a la izquierda como a la derecha.

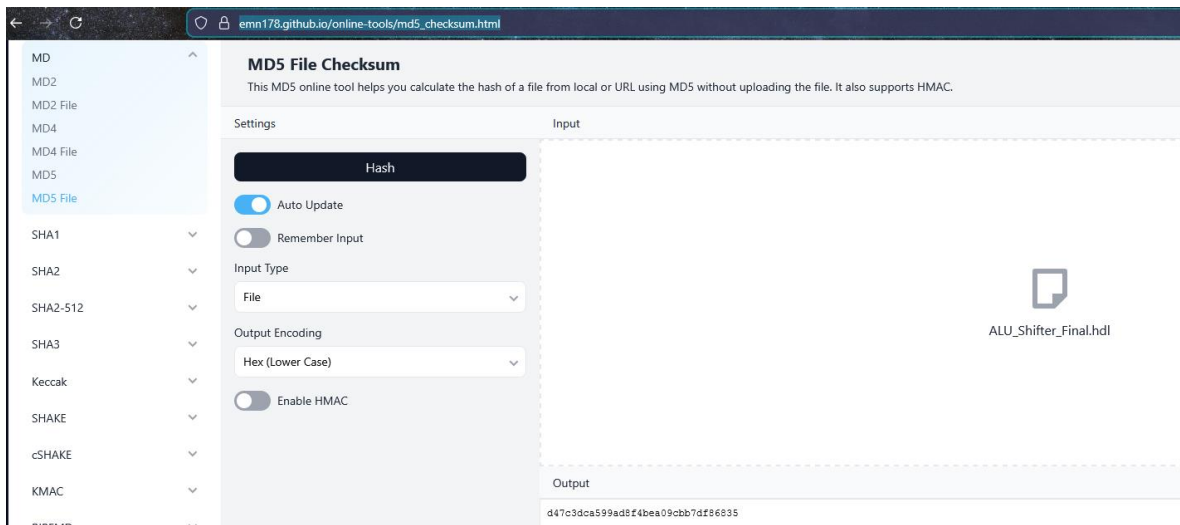
Condiciones a validar Use los casos de prueba mostrados en la tabla 1 para validar que el circuito funciona correctamente.

Entregables Subir a su repositorio de GIT en la carpeta proyecto2 los siguientes archivos:

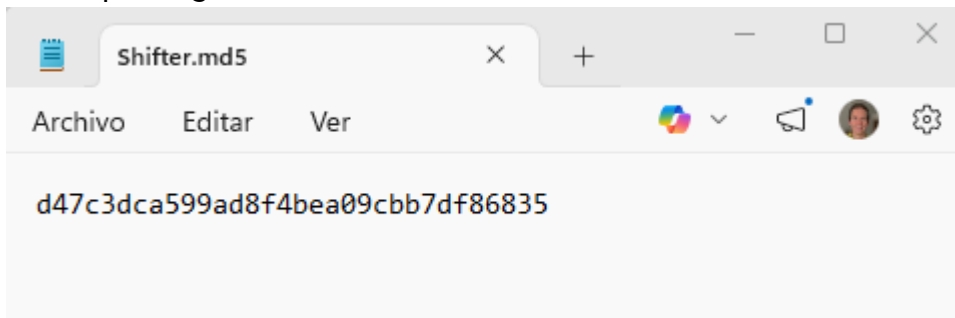
1. Shifter.hdl con el circuito del Shifter.
2. Crear un archivo llamado Shifter.md5 con la salida de la siguiente página web donde debe subir el archivo una vez esté listo:

https://emn178.github.io/online-tools/md5_checksum.html

Por ejemplo, usted va a subir el archivo (en la pantalla se ve uno con nombre ALU_Shifter_Final.hdl). Al entrar al sitio, usted sube el archivo en el campo Input el sitio y en el Output genera la siguiente salida:



Ese output se guarda así en el archivo llamado Shifter.md5



Dicho archivo se sube al repositorio en la carpeta proyecto2. Finalizando el punto 1, su carpeta debería lucir así:

```
[suUsuarioGit]/
├── organizacionComputadores/
│   ├── proyecto2/
│   │   ├── Shifter.hdl
│   │   ├── Shifter.md5
│   │   ├── CONTRIBUTORS.md
│   │   ├── CHANGELOG.md
│   │   ├── LICENSE
│   └── README.md
└── # Colaboradores y roles
    # Historial de cambios
    # Licencia del proyecto
```

Punto # 2 - ALU con Shifter (25%)

Herramientas a utilizar Nand2Tetris

Descripción Agregar a la ALU que ya construyó, la implementación del comando Shifter.

Este comando se reconocerá en la ALU de la siguiente manera:

zx=0, nx=0, zy=0, ny=0, no=1. La función f determinará si el shift es a la izquierda (0) o a la derecha (1).

La nueva especificación de la ALU adaptada para este comando es la siguiente:

```
/******
* ALU.hdl - ALU (Arithmetic Logic Unit):
* Computes out = one of the following functions:
*           0, 1, -1, x, y, !x, !y, -x, -y,
*           x + 1, y + 1, x - 1, y - 1,
*           x + y, x - y, y - x,
*           x & y, x | y
* Autor 1:
* Autor 2:
* *****/
CHIP ALU {
    IN
        x[16], y[16], // 16-bit inputs
        zx, // zero the x input?
        nx, // negate the x input?
        zy, // zero the y input?
        ny, // negate the y input?
        f, // compute (out = x + y) or (out = x & y)? or
           // ((shift left if (zx=nx=zy=ny=0 and f = 0 and
           // no = 1)) or (shift right if (zx=nx=zy=ny=0 and
           // f = 1 and no = 1)))
        no; // negate the out output?
    OUT
        out[16], // 16-bit output
        zr,      // if (out == 0) equals 1, else 0
        ng,      // if (out < 0) equals 1, else 0
        result;  // if (shift left) equals in[15]
                // if (shift right) equals in[0]
    PARTS:
        //// Replace this comment with your code.
}
```

Condiciones a validar Use los casos de prueba mostrados en la tabla 1 para validar que el circuito funciona correctamente *de manera manual*.
Adicionalmente, valide las pruebas unitarias de la ALU en la plataforma Nand2Tetris *deben funcionar correctamente*.

Entregables Subir a su repositorio de GIT en la carpeta proyecto2 los siguientes archivos:

1. Archivo ALU.hdl donde se implemente la ALU que incluye el llamado al chip Shifter implementado en el punto 1, probada y funcionando.
2. Genere el archivo ALU.md5 tal y como generó el archivo en el punto 1.

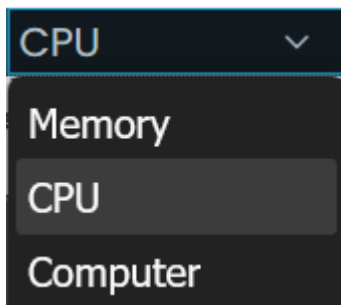
Finalizando el punto 2, su carpeta debería lucir así:

```
[suUsuarioGit]/
├── organizacionComputadores/
│   ├── proyecto2/
│   │   ├── ALU.hdl
│   │   ├── ALU.md5
│   │   ├── Shifter.hdl
│   │   └── Shifter.md5
│   ├── CONTRIBUTORS.md          # Colaboradores y roles
│   ├── CHANGELOG.md            # Historial de cambios
│   ├── LICENSE                  # Licencia del proyecto
│   └── README.md
```

Punto # 3 - Implementación de la CPU (40%)

Herramientas a utilizar Nand2Tetris

Descripción Implemente la CPU del Nand2Tetris en HDL implementado los circuitos del proyecto 5:



Consulte en la página del simulador de Nand2Tetris

<https://nand2tetris.github.io/web-ide/chip> en el Project 5 la descripción de los chips mas lo visto en las clases 12, 13 y 14 y en el capítulo 5 del libro <https://b1391bd6-da3d-477d-8c01->

38cdf774495a.filesusr.com/ugd/44046b_b2cad2eea33847869b86c541683551a7.pdf
f

Condiciones a validar Valide las pruebas unitarias de los chips Memory, CPU y Computer. En la plataforma Nand2Tetris *deben funcionar correctamente*.

Entregables Subir a su repositorio de GIT en la carpeta proyecto2 los siguientes archivos:

1. Archivo Memory.hdl donde se implementen la memoria RAM.
2. Archivo CPU.hdl que implementa el diagrama de la CPU visto en clase, que incluye el registro A, D, la ALU implementada en el punto 2 y el registro PC.
3. Archivo Computer.hdl que implementa la CPU, el llamado al chip Memory (entregable 1 punto 3) y la ROM de 32K.
4. Genere los archivos md5 de cada HDL generado (Memory.md5, CPU.md5 y Computer.md5) tal y como se generaron en los puntos anteriores.

Finalizando el punto 3, su carpeta debería lucir así:

```
[suUsuarioGit]/
├── organizacionComputadores/
│   ├── proyecto2/
│   │   ├── ALU.hdl
│   │   ├── ALU.md5
│   │   ├── Computer.hdl
│   │   ├── Computer.md5
│   │   ├── CPU.hdl
│   │   ├── CPU.md5
│   │   ├── Memory.hdl
│   │   ├── Memory.md5
│   │   ├── Shifter.hdl
│   │   └── Shifter.md5
│   ├── CONTRIBUTORS.md          # Colaboradores y roles
│   ├── CHANGELOG.md            # Historial de cambios
│   ├── LICENSE                  # Licencia del proyecto
│   └── README.md
```

Punto # 4 - Arquitectura de la instrucción (10%)

Descripción Diseñe la instrucción tipo C para que se pueda generar el binario de los comandos shift left y shift right.

Recuerden que las instrucciones tipo C tienen el siguiente formato:

111 a cccccc ddd jjj

La instrucción Shift Left tiene la siguiente sintaxis:

dest=reg << 1

Un ejemplo del assembler del comando es:

D=M<<1

La instrucción Shift Right tiene la siguiente sintaxis:

dest=reg >> 1

Un ejemplo del assembler del comando es:

AM=D>>1

Entregables Subir a su repositorio de GIT en la carpeta proyecto2 el siguiente archivo:

1. design.txt donde se describe cómo se traducirá en bits los comandos << y >>. Use el mismo encabezado de los puntos anteriores.
2. Genere el archivo design.md5 tal y como se generaron en el archivo en los puntos anteriores.

Finalizando el punto 4, su carpeta debería lucir así:

```
[suUsuarioGit]/
├── organizacionComputadores/
│   ├── proyecto2/
│   │   ├── ALU.hdl
│   │   ├── ALU.md5
│   │   ├── Computer.hdl
│   │   ├── Computer.md5
│   │   ├── CPU.hdl
│   │   ├── CPU.md5
│   │   ├── Memory.hdl
│   │   ├── Memory.md5
│   │   ├── Shifter.hdl
│   │   ├── Shifter.md5
│   │   ├── design.txt
│   │   └── design.md5
│   └── CONTRIBUTORS.md
└── # Colaboradores y roles
```

| CHANGELOG.md
 | LICENSE
 | README.md

Historial de cambios
 # Licencia del proyecto

Rúbrica Proyecto 2 --> Fecha de entrega 29 de abril

Rango	Criterio				
	Aplicación de conceptos	Conocimiento	Cumplimiento requisitos	Funcionamiento	Optimización
	20%	20%	20%	20%	20%
4.5 - 5.0	Aplica correctamente ALU, CPU y assembler integrados	Justifica decisiones de diseño con claridad técnica	Todos los archivos HDL y MD5 correctos	Todos los tests Nand2Tetris pasan	Arquitectura modular limpia (bonificación)
3.5 - 4.4	Implementa correctamente pero con leves redundancias	Explicaciones parciales o incompletas	Entrega completa con errores menores	Funciona en la mayoría de pruebas	Aceptable modularidad
0.0 - 3.4	Errores conceptuales evidentes	No logra justificar decisiones	Faltan entregables	No pasan pruebas unitarias	No evaluada

Cada punto tendrá una nota de acuerdo con el rango y se multiplica por el porcentaje de cada punto. La suma de cada una genera la nota de cada criterio. La nota definitiva del proyecto es la suma de cada criterio dividido por 5.

Consideraciones especiales

1. Este proyecto es para ser realizado en equipos de dos personas.
2. Se espera calidad profesional
3. Documentación exhaustiva
4. Pruebas completas
5. Código bien estructurado
6. Presentación organizada
7. Evaluación de Contribución Individual
 - a. Auto-evaluación: Cada miembro evalúa su contribución
 - b. Evaluación por pares: Miembros evalúan a sus compañeros
 - c. Evidencia tangible: Commits, issues resueltos, documentación
8. Conflictos en Equipo
 - a. Reportar al profesor antes del 17 de abril si hay problemas
 - b. Posibilidad de redistribución o cambio de modalidad
 - c. Después del 17 de abril, el equipo se mantiene
9. Nota: Trabajar en equipo simula entornos profesionales reales donde la coordinación, comunicación y gestión de proyectos son tan importantes como las habilidades técnicas. La evaluación considerará tanto el producto final como el proceso de desarrollo colaborativo.

Universidad Eafit - Ingeniería de sistemas

Organización de computadores 2026-1 - 25 de marzo de 2026

Proyecto 3 Traductor de instrucciones Assembler Hack y desensamblador de instrucciones binarias Hack.

Docente José Luis Montoya Pareja

Objetivo

Practicar los conceptos vistos en clase durante todo el semestre desde el diseño de una tabla de verdad, ALUs, construcción de la CPU y programación en assembler hasta su implementación en el HDL Nand2Tetris con circuitos adicionales a los que la plataforma Nand2Tetris trae.

Paso inicial

Cree en su repositorio de GIT dentro de la carpeta organizacionComputadores la carpeta proyecto3 y haga un commit inicial con la siguiente estructura:

```
[suUsuarioGit]/
├── organizacionComputadores/
│   ├── proyecto3/
│   ├── CONTRIBUTORS.md          # Colaboradores y roles
│   ├── CHANGELOG.md            # Historial de cambios
│   ├── LICENSE                  # Licencia del proyecto
│   └── README.md
```

En el archivo CONTRIBUTION.md se crea una tabla con la siguiente información:

Contribución de Miembros

Nombre	Rol	Tareas Asignadas	% Contrib
-----	-----	-----	-----
Ana Pérez	Líder	Coordinación, shell loop	20%
Luis Gómez	Dev	Comandos archivos, parser	25%
Carlos Ruiz	Dev	Comandos sistema, utils	25%
María López	Tester	Pruebas, depuración	20%
Pedro Díaz	Doc	Documentación, ejemplos	10%
Total:			100%

En el archivo README.md incluya la siguiente información:

****Organización de computadores****

****Proyecto 3****
**** 2026-1****
****Integrantes proyecto 3****
[Nombre alumno 1]
[Nombre alumno 2]

Punto # 1 - Programa traductor (60%)

Herramientas a utilizar Lenguaje de programación (Java o Python)

Descripción Debido a que agregamos las instrucciones del shift tanto left como right al computador Hack y a su assembler, el traductor que trae la plataforma de Nand2tetris no es funcional al 100%. Hay que incorporar a la traducción del assembler completo la del comando shift y generar el assembler para que pueda ser ejecutado en la computadora que crearon con el HDL.

Debe diseñar una solución en Java o Python que pueda leer un archivo con extensión .asm (por ejemplo, Prog.asm) que es un programa de texto que tiene una secuencia de líneas donde cada una es un string que representa una línea vacía (un enter), un comentario de una línea (representado por //), una instrucción tipo A o tipo C, o una etiqueta.

La salida debe ser un archivo con extensión .hack (por ejemplo, Prog.hack) que es un archivo tipo texto que contiene una secuencia de líneas, cada una representa un string de 16 caracteres 0 ó 1.

Uso: (por ejemplo, si se implementa en Java):

\$ java HackAssembler Prog.asm

Acción: Crea un archivo llamado Prog.hack, que contiene el programa traducido al binario de Hack. El nombre del programa se recibe como parámetro de la ejecución del programa HackAssembler.

Si el archivo de entrada no tiene errores en la sintaxis del archivo, el archivo se traduce por completo y no muestra ningún mensaje en pantalla.

Si se encuentra algún error durante el proceso, el programa genera un mensaje en pantalla indicando la línea del archivo de entrada donde se identificó el error, se cierra el archivo de salida y se detiene el proceso de traducción.

Condiciones a validar El profesor suministrará después de semana santa los casos de prueba (que incluye la instrucción Shift) para generar la traducción. Luego de traducido el programa a binario, se debe cargar en la CPU creada por ustedes y

ejecutar para validar que todo funcione acorde. Esta validación se enseñará en clase cómo hacerlo.

Entregables Subir a su repositorio de GIT en la carpeta proyecto3 los siguientes archivos:

1. Los archivos fuente (java o py) con el programa traductor. Los archivos de cada clase deben tener el siguiente encabezado **obligatorio** (puede copiarlo y pegarlo en el archivo HDL para desarrollarlo) incluyendo los nombres de los autores.

```
/*  
 * Clase.java - Descripción general de lo que hace la clase  
 * Autor 1:  
 * Autor 2:  
 */
```

2. Crear un archivo md5 por cada archivo .java usado en el programa con la salida de la siguiente página web donde debe subir el archivo una vez esté listo: https://emn178.github.io/online-tools/md5_checksum.html (como se hizo en el proyecto 2).

Finalizando el punto 1, su carpeta debería lucir así como mínimo:

```
[suUsuarioGit]/  
├── organizacionComputadores/  
│   ├── proyecto3/  
│   │   ├── HackAssembler/  
│   │   │   ├── docs/  
│   │   │   │   ├── API.md          # Documentación de las clases  
│   │   │   │   ├── DESIGN.md       # Diagrama de clases  
│   │   │   │   └── USER_GUIDE.md   # Cómo ejecutar el programa  
│   │   │   ├── src/  
│   │   │   │   ├── HackAssembler.java  
│   │   │   │   └── HackAssembler.md5  
│   │   │   ├── test/  
│   │   │   │   ├── HackAssemblerTest.java  
│   │   │   │   └── HackAssemblerTest.md5  
│   │   │   └── README.md  
│   ├── CONTRIBUTORS.md             # Colaboradores y roles  
│   ├── CHANGELOG.md                # Historial de cambios  
│   └── LICENSE                      # Licencia del proyecto
```

_____ README.md

Use de preferencia Maven para la compilación en java. Suba al repositorio los archivos necesarios para la compilación evitando incluir los archivos .class y .jar en el repositorio.

Punto # 2 - Clase desensamblador (40%)

Herramientas a utilizar Lenguaje de programación (Java o Python)

Descripción Agregar una clase que lea el archivo con extensión .hack (por ejemplo, Prog.hack) que es un archivo tipo texto que contiene una secuencia de líneas, cada una representa un string de 16 caracteres 0 ó 1 y traduzca cada línea a su instrucción en assembler tipo A o tipo C correspondiente.

Uso: (por ejemplo, si se implementa en Java):

```
$ java HackAssembler -d Prog.hack
```

Acción: Crea un archivo llamado ProgDis.asm, que contiene el programa traducido desde binario de Hack al assembler. El nombre del programa se recibe como parámetro de la ejecución del programa HackAssembler junto con el flag -d que le indica al programa que el programa que va a leer es un binario de hack.

Si el archivo de entrada no tiene errores en la sintaxis del archivo, el archivo se traduce por completo y no muestra ningún mensaje en pantalla.

Si se encuentra algún error durante el proceso, el programa genera un mensaje en pantalla indicando la línea del archivo de entrada donde se identificó el error, se cierra el archivo de salida y se detiene el proceso de traducción.

Condiciones a validar El profesor suministrará después de semana santa los casos de prueba (que incluye la instrucción Shift) para generar la traducción del desensamblado. Luego de traducido el programa a assembler, se debe validar que el programa si haga lo mismo que el programa original. Esta validación se enseñará en clase cómo hacerlo.

Entregables Subir a su repositorio de GIT en la carpeta proyecto3 los siguientes archivos:

1. Los archivos fuente (java o py) con la clase HackDisassembler. El archivo de la clase debe tener el siguiente encabezado **obligatorio** (puede copiarlo y pegarlo en el archivo HDL para desarrollarlo) incluyendo los nombres de los autores.

```

/*****
* HackDisassembler.java - Descripción de lo que hace la clase
* Autor 1:
* Autor 2:
*****/

```

2. Crear un archivo md5 por cada archivo .java usado en el programa con la salida de la siguiente página web donde debe subir el archivo una vez esté listo: https://emn178.github.io/online-tools/md5_checksum.html (como se hizo en el proyecto 2).

Finalizando el punto 2, su carpeta debería lucir así como mínimo:

```

[suUsuarioGit]/
├── organizacionComputadores/
│   ├── proyecto3/
│   │   ├── HackAssembler/
│   │   │   ├── docs/
│   │   │   │   ├── API.md          # Documentación de las clases
│   │   │   │   ├── DESIGN.md       # Diagrama de clases
│   │   │   │   └── USER_GUIDE.md   # Cómo ejecutar el programa
│   │   │   ├── src/
│   │   │   │   ├── HackAssembler.java
│   │   │   │   ├── HackAssembler.md5
│   │   │   │   ├── HackDisassembler.java
│   │   │   │   └── HackDisassembler.md5
│   │   │   ├── test/
│   │   │   │   ├── HackAssemblerTest.java
│   │   │   │   └── HackAssemblerTest.md5
│   │   │   └── README.md
│   ├── CONTRIBUTORS.md             # Colaboradores y roles
│   ├── CHANGELOG.md               # Historial de cambios
│   ├── LICENSE                    # Licencia del proyecto
│   └── README.md

```

Use de preferencia Maven para la compilación en java. Suba al repositorio los archivos necesarios para la compilación evitando incluir los archivos .class y .jar en el repositorio.

Rúbrica Proyecto 3 --> Fecha de entrega 29 de abril

Criterio	Peso (%)	Excelente (4.5–5.0)	Bueno (3.5–4.4)	Aceptable (2.5–3.4)	Deficiente (0.0–2.4)
Aplicación de los conceptos (ISA Hack)	20	Implementa correctamente la traducción <code>assembler</code> ↔binario, incluyendo instrucciones <code>shift</code> , símbolos y etiquetas.	Traduce correctamente la mayoría de los casos, con detalles menores imprecisos.	Traducción parcial; errores en algunas instrucciones o símbolos.	Errores conceptuales graves en el proceso de traducción.
Conocimiento y justificación del diseño	20	Justifica claramente decisiones de parsing, manejo de tablas de símbolos y diseño del desensamblador.	Justificaciones correctas pero incompletas.	Justificaciones superficiales o repetición del enunciado.	No justifica ni demuestra comprensión del diseño.
Cumplimiento del enunciado y entregables	20	Entrega completa con estructura solicitada, documentación y archivos <code>md5</code> correctos.	Entrega completa con errores menores de forma.	Faltan entregables o hay inconsistencias evidentes.	Incumplimiento significativo del enunciado.
Funcionamiento del traductor y desensamblador	20	Ambos programas funcionan correctamente y preservan la semántica del programa.	Funcionan en general pero con fallos controlados.	Funcionamiento parcial; fallan varios casos.	El sistema no funciona correctamente.
Diseño de software y modularidad (criterio diferenciador)	20	Código modular, claro, con separación de responsabilidades; puede otorgar bonificación.	Diseño aceptable con redundancias.	Diseño básico poco mantenible.	Código desordenado y monolítico.

Cada punto tendrá una nota de acuerdo con el rango y se multiplica por el porcentaje de cada punto. La suma de cada una genera la nota de cada criterio. La nota definitiva del proyecto es la suma de cada criterio dividido por 5.

Consideraciones especiales

- Este proyecto es para ser realizado en equipos de dos personas.
- Se espera calidad profesional
- Documentación exhaustiva
- Pruebas completas
- Código bien estructurado
- Presentación organizada
- Evaluación de Contribución Individual
 - Auto-evaluación: Cada miembro evalúa su contribución
 - Evaluación por pares: Miembros evalúan a sus compañeros
 - Evidencia tangible: Commits, issues resueltos, documentación
- Conflictos en Equipo
 - Reportar al profesor antes del 17 de abril si hay problemas
 - Posibilidad de redistribución o cambio de modalidad
 - Después del 17 de abril, el equipo se mantiene
- Nota: Trabajar en equipo simula entornos profesionales reales donde la coordinación, comunicación y gestión de proyectos son tan importantes como las habilidades técnicas. La evaluación considerará tanto el producto final como el proceso de desarrollo colaborativo.